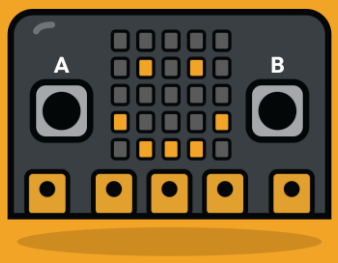
DEVOXX4KIDS · WORKSHOP

Micro:bit – Pong

Recreate one of the oldest computer games, and control the paddle by tilting your micro:bit.

Scratch | micro:bit | Ages 8–14



What you need

Scratch editor	Hex file for the micro:bit	The code for this project
This is where you write the code scratch.mit.edu/projects/editor	Download it and copy it onto your micro:bit scratch.mit.edu/microbit	Handy if you need this assignment again later scratch.mit.edu/projects/362143391

In this project, we are going to recreate one of the oldest computer games: **Pong**.

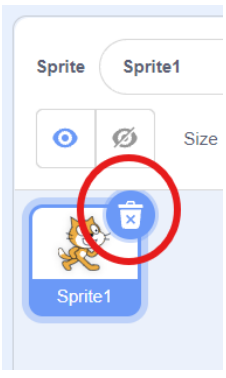
In that game, you try to bounce a ball back and forth across the playing field using a paddle. We will make a single-player version of this game, where you control the paddle with the micro:bit. We will write the code for this assignment in Scratch.

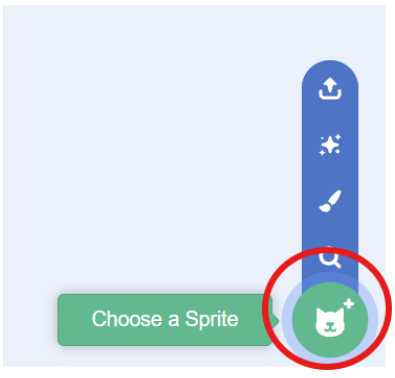
1 Bouncing ball

In Scratch, you work with "things" that move around the stage. These "things" are called **sprites**.

In this game, we start with a ball that bounces across the screen.

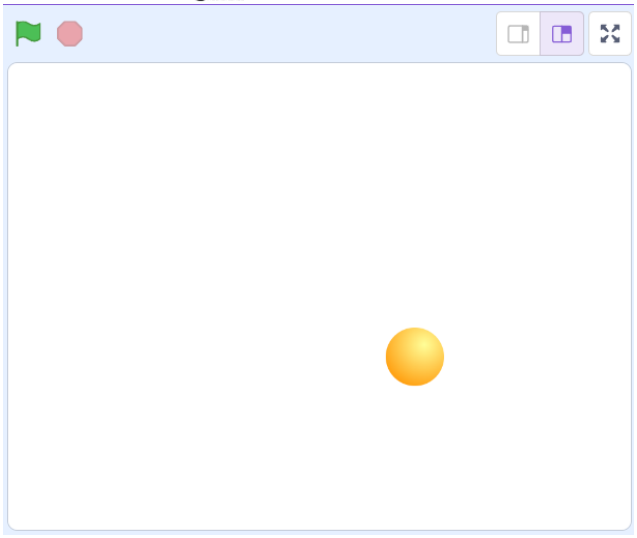
Delete the cat sprite by clicking the trash can icon, and create a new sprite by clicking **Choose a Sprite** and selecting the ball:







You now have one ball on the stage:



We're going to make that ball bounce.

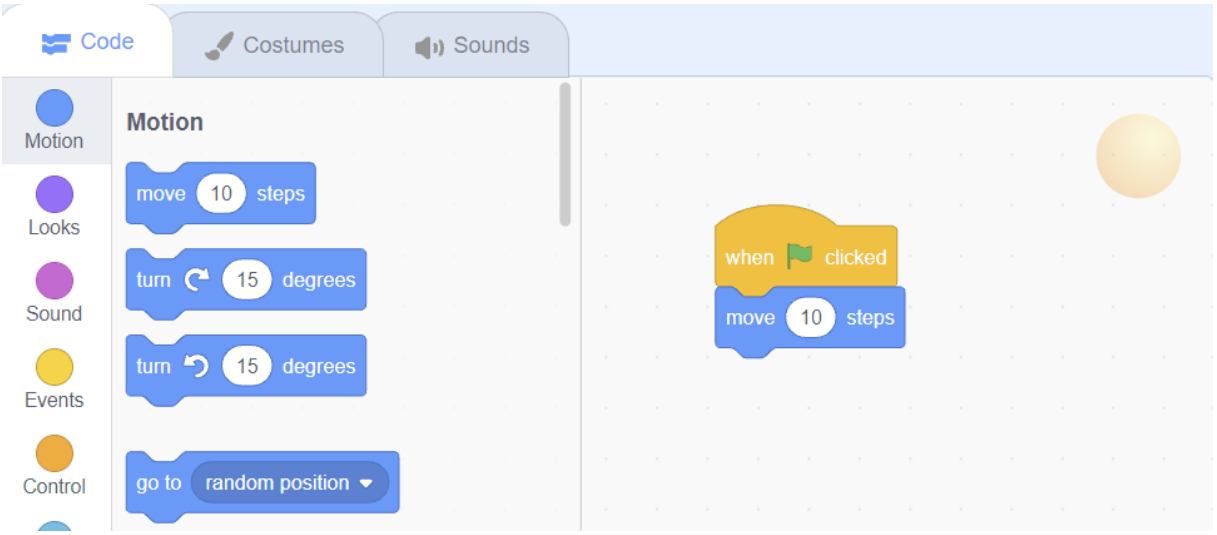
Under the **Code** tab, you can program the behaviour of sprites. That behaviour consists of small steps, described with blocks that you connect. Each type of behaviour has its own colour. Everything related to motion is blue, and so on.

Drag 

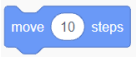
(from 

) and 

onto the large white area and snap them together:

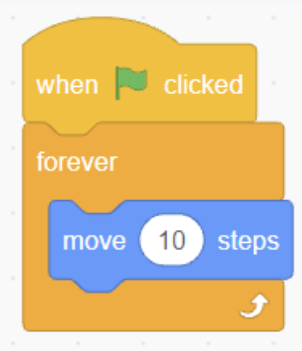


Test your program by clicking the green flag above the stage. If everything is correct, the ball should move slightly to the right.

Of course, we don't want to keep clicking the green flag, so we place a  loop around the 

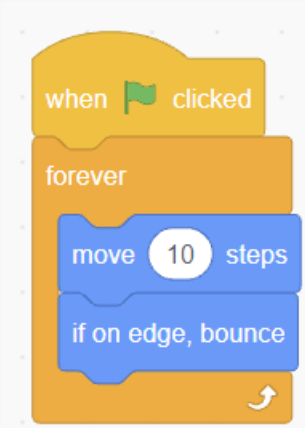
block. Adjust your script so it looks like this:

SCRATCH CODE



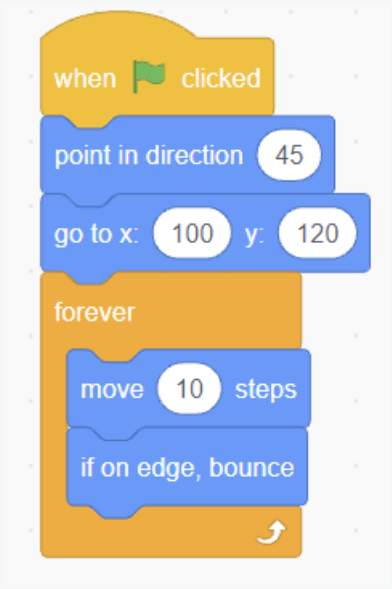
Test your program. The ball gets stuck on the right side. The solution: **bounce back on the edge**!

SCRATCH CODE



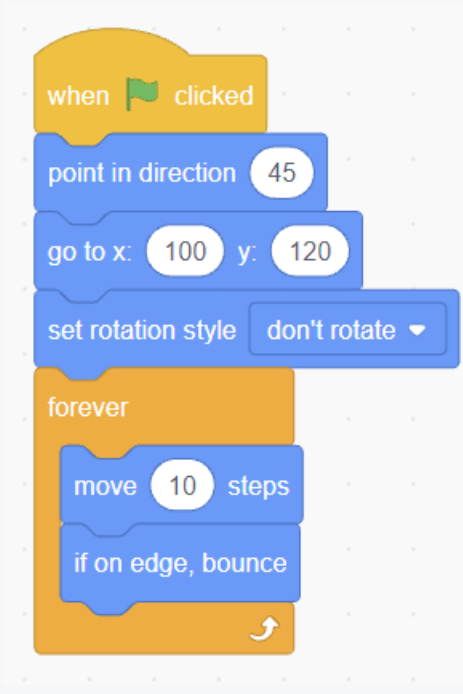
Now the ball moves left and right, but we want it to move across the entire playing field, so we give it a different direction. The ball may always start in the same position:

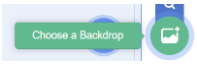
SCRATCH CODE



When bouncing, the shadow of the ball rotates. That doesn't look very nice. By adjusting the rotation style, the shadow stays in place:

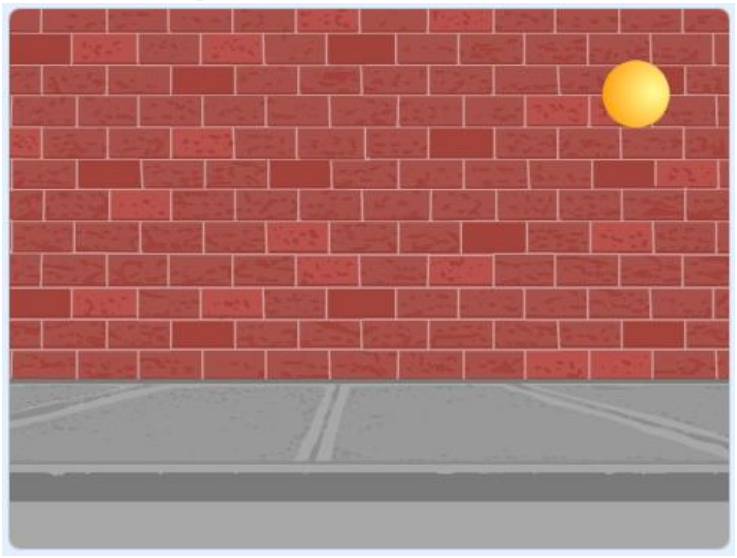
SCRATCH CODE



The ball now bounces back and forth across a white playing field. That can be more fun: click 

Choose a Backdrop and choose the **"Wall 1"** backdrop.

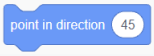
You should now see this (with a moving ball):



Save This is a good moment to save your program. Click **File** → **Save to your computer**. Give it a name if asked.

A bit of maths...

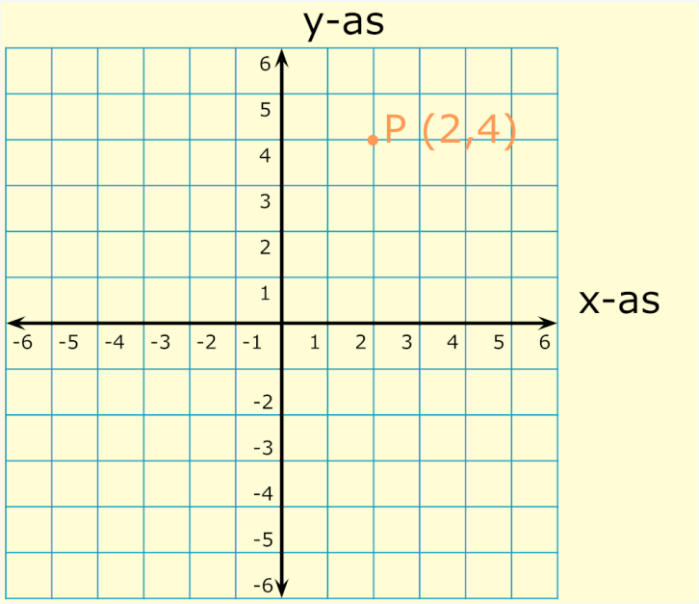
You may skip this part, but if you want to know what the numbers in 

and 

mean, we explain it here.

Coordinates

In Scratch, every sprite has a position on the screen. That position is defined by two numbers: how far left or right (the x-axis) and how far up or down (the y-axis).



In this image, the orange point P is 2 steps to the right and 4 steps up from the origin. The x-value of point P is 2, the y-value is 4. In mathematics, this is written as P (2,4).

In Scratch, the x-axis runs from -240 to 240 and the y-axis from -180 to 180.

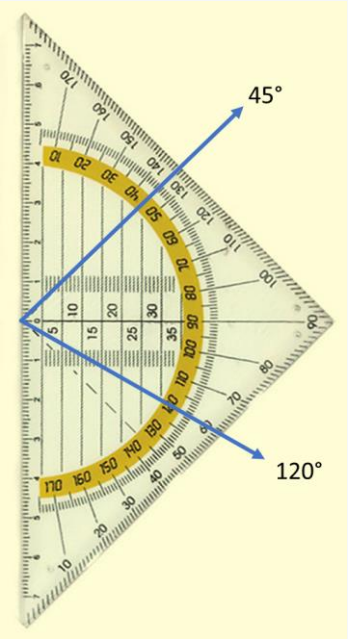
Direction

You've probably seen a protractor before. It has a half-circle with numbers on it. Those numbers indicate direction. When you make a sprite move in Scratch, the direction determines where it goes.

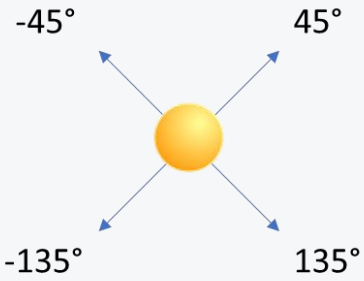
Here you see, for example, a direction of 45 degrees and one of 120 degrees.

The numbers on a protractor go from 0 to 180, but you can extend this to a full circle: 360 degrees.

The numbers can also be negative, meaning the rotation goes anticlockwise instead of clockwise.



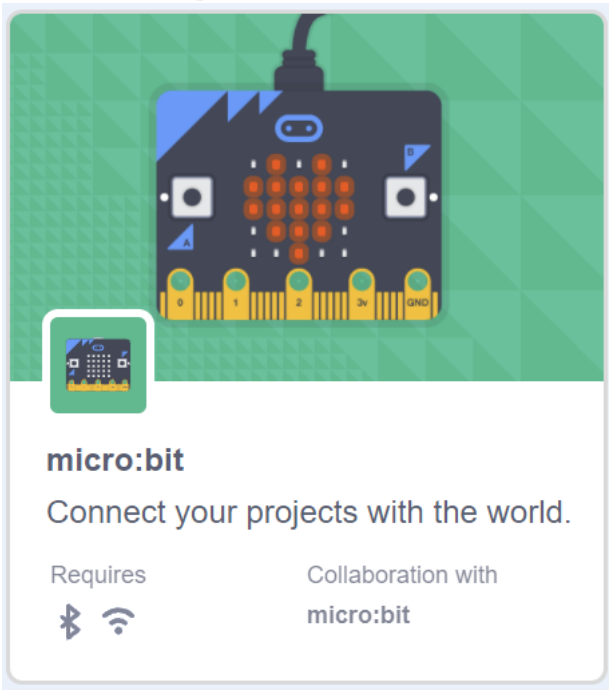
When the ball bounces, it changes direction. In this assignment, you will encounter four different directions:



2 Connecting the micro:bit

To connect the micro:bit:

1. First, look at the micro:bit and check which letters appear on the display.
2. Then, in Scratch, click **Add Extension**  in the bottom left and select the **micro:bit** tile.



You will now see a list of micro:bits. Choose the one with the correct letters (in this case "vovot") and then click **Go to Editor**.

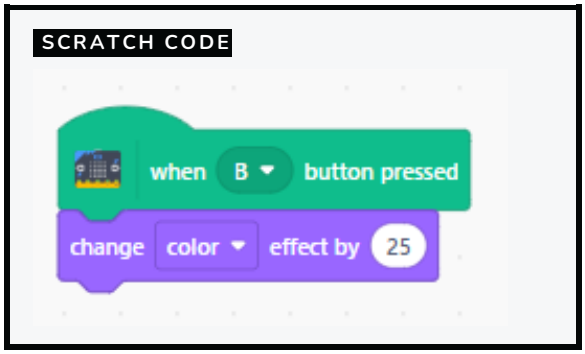


Not working? If no micro:bits appear, Scratch Link may not be installed, or the hex file is not on the micro:bit. Click **Help** and follow the instructions.

The micro:bit may lose its connection during this assignment. You can reconnect it from this screen.

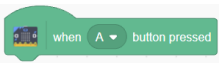
3 Controlling the game with the micro:bit

First, a quick test to see if the micro:bit works. Write this code:



Press the right button on the micro:bit and check whether the ball changes colour.

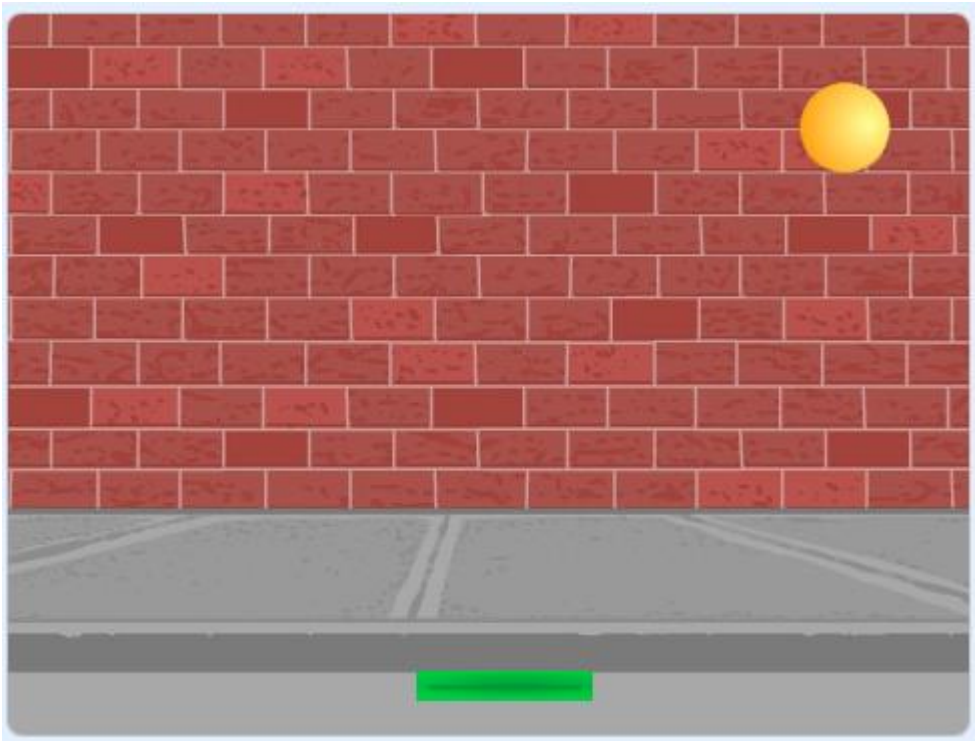
Let's start the game by pressing a button on the micro:bit. Replace 

with 

so the game starts when you press the left button.

4 The paddle

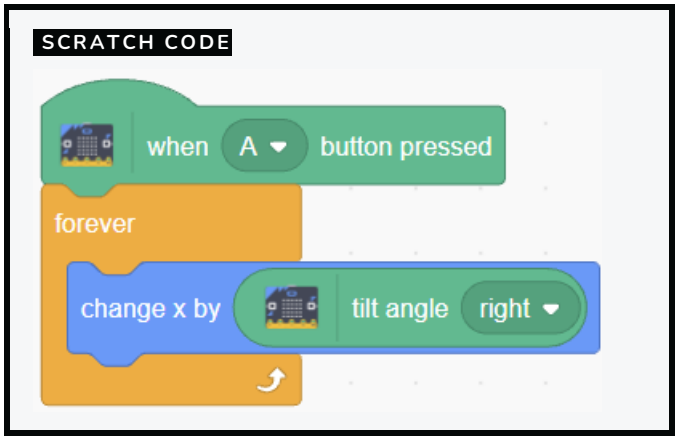
Add a new **Paddle** sprite and place it centred at the bottom of the stage:



When you add this sprite, it may look like your code disappeared. That's because each sprite has its own code. The code for the ball is still under the ball sprite.

Besides the two buttons, the micro:bit can also detect tilt. We can use that to control the paddle. Add the following code to the paddle sprite:

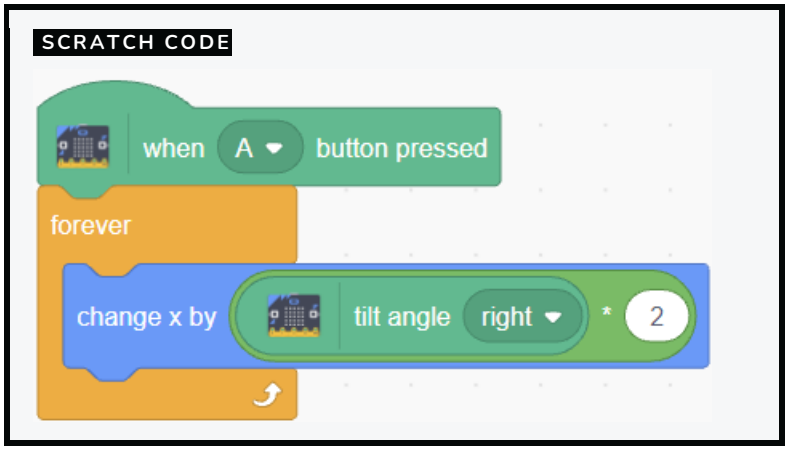
SCRATCH CODE



When you tilt the micro:bit, you can move the paddle left and right.

... except the paddle doesn't move far enough. If you multiply the tilt angle by 2, the paddle will move across the entire screen:

SCRATCH CODE

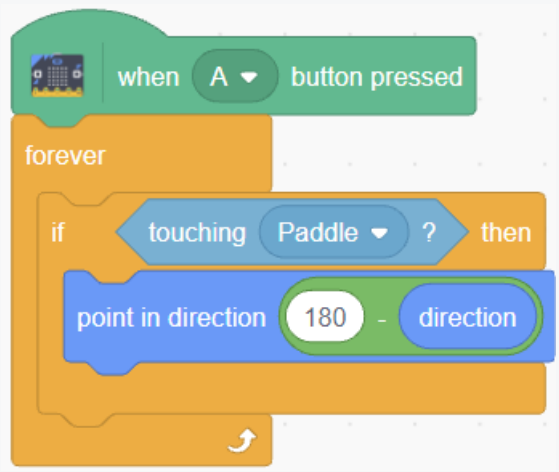


5 Bouncing off the paddle

Go back to the **Ball** sprite.

Keep the two scripts you already made. Create a new script that makes the ball bounce off the paddle:

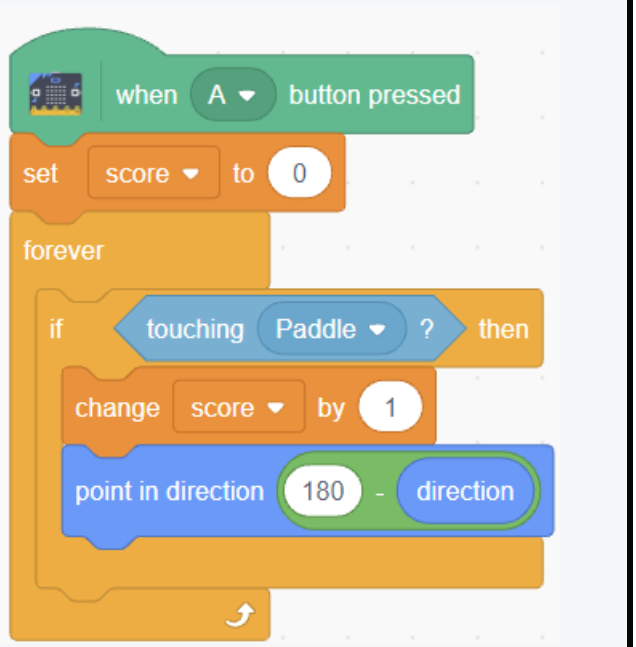
SCRATCH CODE



This is a good place in the code to keep track of the score. Create a new variable (for all sprites) and call it **score**.

At the start of the game, set the score to 0, and each time the ball hits the paddle, add 1 point:

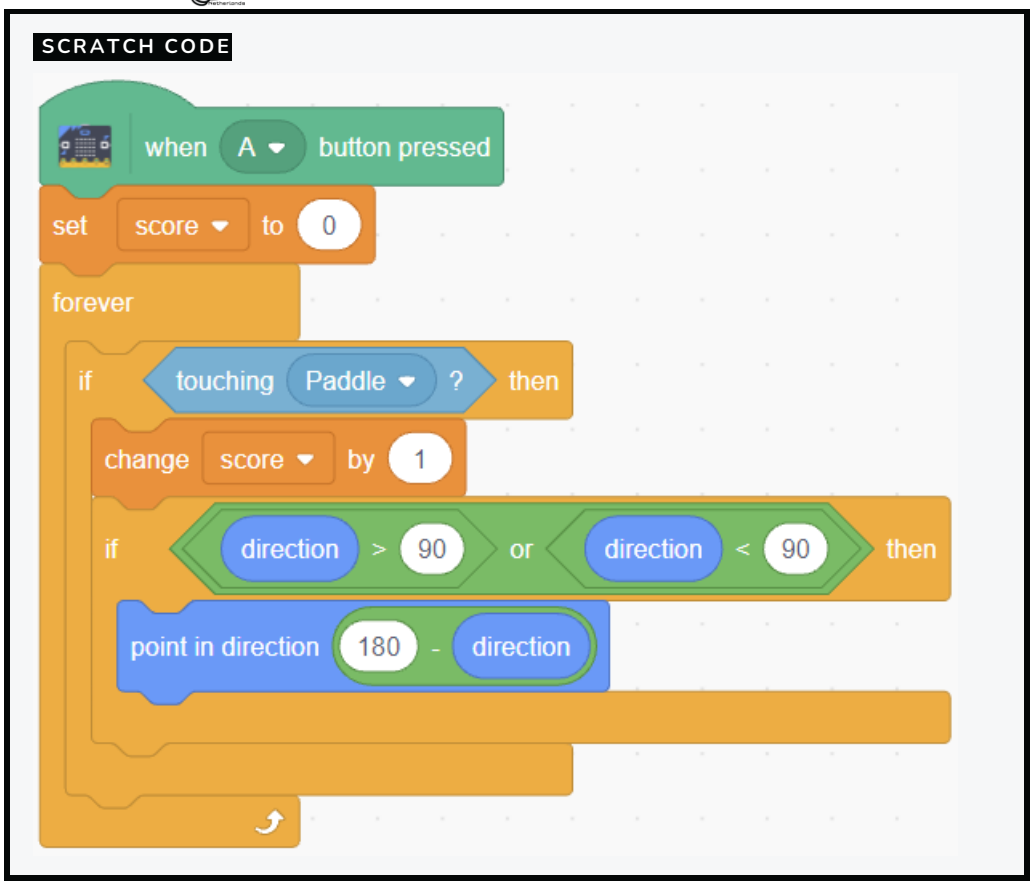
SCRATCH CODE



Save Don't forget to save your code occasionally.

Does the game work a bit already? Play it for a while and see what you notice.

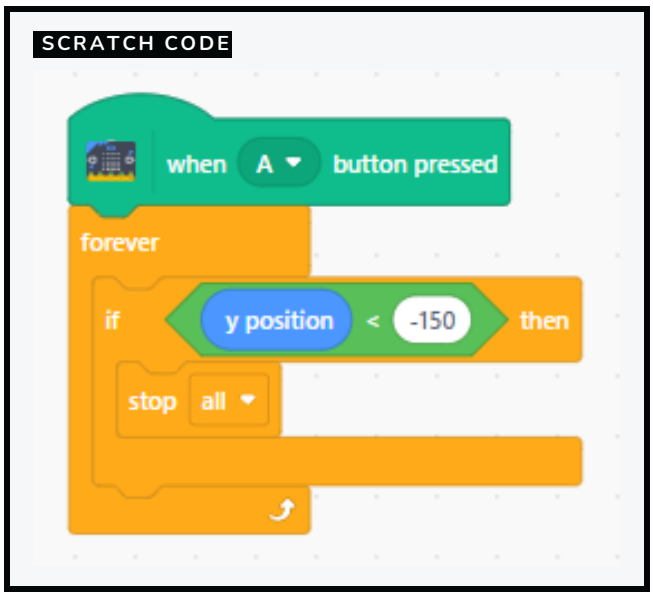
If you hit the ball with the side of the paddle, it sometimes keeps bouncing "inside" the paddle. If you want to prevent that, you can make the ball bounce only if it was moving downward before:



6 Game over

If you test the game now, you can score points by bouncing the ball correctly. But if you miss the ball, the game just continues. That's not what we want.

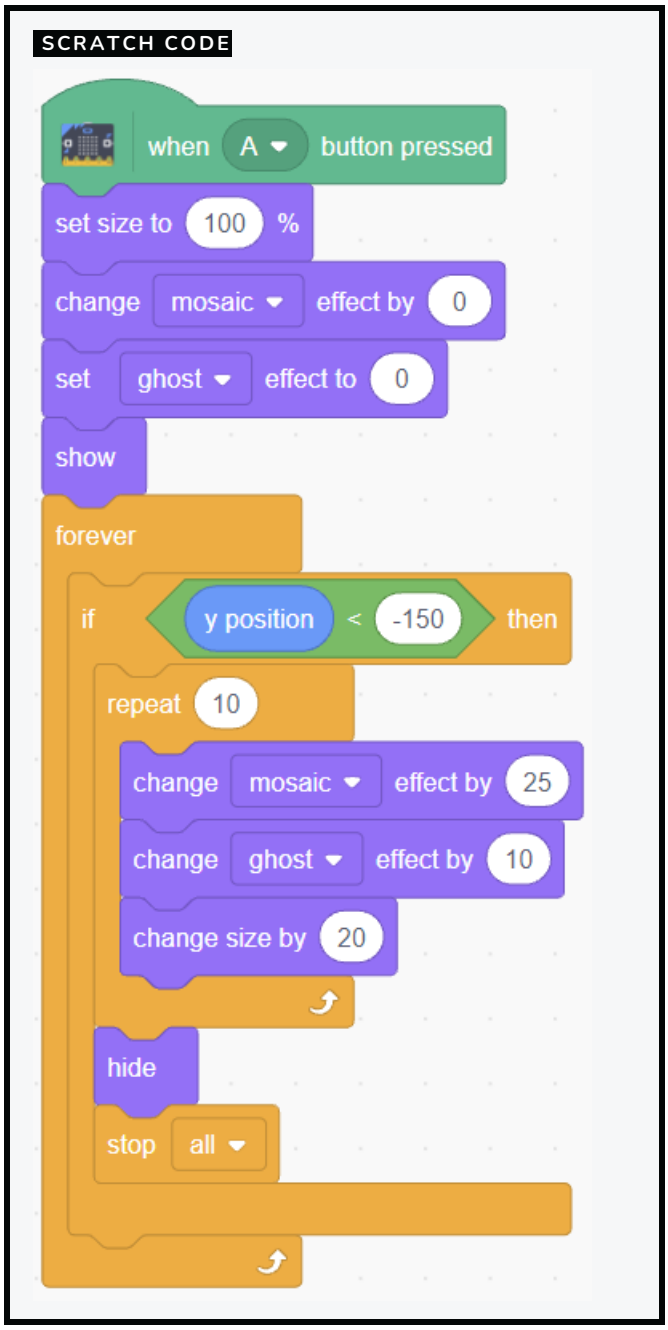
Add a new script to stop the game when the ball goes too low:



Try the game.

Game over can be much more fun! By adding effects, you can make the ball explode or fade out.

Make sure you reset all sprite effects at the start of the game, otherwise the game won't work properly the second time. For example:



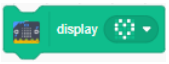
I don't need to remind you to save your code from time to time.

Let someone else play the game too. They might have good suggestions.

7 Further extensions

How could you improve the game? Think about:

- Sounds
- Making the ball go faster as you score more points

- The micro:bit also has a 

block. What could you do with that? Or maybe display text?

Let your imagination run wild, make something great!

... and if you get stuck: asking for help is always allowed!

8 Installation tips (for home use)

In this assignment, we use a connection between Scratch and the micro:bit. This connection has two parts: one on the computer (or tablet) and one on the micro:bit.

- For the computer part, you need a separate program called **Scratch Link** on Windows and macOS. On Android and ChromeOS, this part is already built into the Scratch app.
- For the micro:bit part, you need a **hex file**, which you copy onto the micro:bit.

These files, installation instructions, and solutions to common problems can be found at scratch.mit.edu/microbit.

The code for this project can be found at scratch.mit.edu/projects/362143391.